CTF Challenge Walkthrough: BetYouCan’tGetIn

Author: Karthik | Difficulty: Hard

This challenge is a web exploitation puzzle that involves source code analysis, directory brute-forcing, and a final XSS attack to retrieve the flag.

Step-by-Step Solution

Step 1: Initial Reconnaissance

Navigate to the Target: You are given the URL: `http://localhost:9000`.

Initial Observation: A complete blank screen. Viewing the page source reveals no obvious comments or hidden HTML elements. Standard reconnaissance yields nothing.

Step 2: Directory Brute-Forcing

First Gobuster Run: Since the main page is blank, the next step is to discover hidden directories or files. Using a tool like `gobuster` is essential.

```
gobuster dir -u http://localhost:9000 -w /usr/share/wordlists/dirb/common.tx
```

Finding: The scan reveals a directory: `/nothing`.

Investigate `/nothing`: Browsing to `http://localhost:9000/nothing` again shows a blank, uninteresting page.

Second Gobuster Run: To be thorough, you run `gobuster` again, this time targeting the `/nothing` directory.

```
gobuster dir -u http://localhost:9000/nothing/ -w /usr/share/wordlists/dirb/
```

Finding: This scan uncovers another directory: `/something`.

Step 3: The Hidden Message

Investigate `/something`: You navigate to `http://localhost:9000/nothing/something`. The page appears blank, but this time, inspecting the page source reveals a clue: the HTML body is filled with what looks like empty space .

Recognize the Encoding: A seasoned CTF player will recognize this pattern as **Whitespace encoding**. The sequence of spaces and tabs can represent a hidden message encoded in a binary or ternary format (like Space, Tab, Line-Feed encoding).

Decode the Whitespace: You copy the "invisible" text from the source code and use an online decoder or a script to decode it.

Tool: You can use CyberChef with the "From Whitespace" recipe.

Result: After decoding, you get the text: `letmein`.

Step 4: Discovering the Secret Endpoint

Searching for a Clue: With the key `letmein` but no place to use it, you spend time investigating other aspects of the application. This involves checking HTTP headers, cookies, and robots.txt.

The Secret Header: You discover that the server responds to a custom header: `X-referer`. This can be found by intercepting requests with a proxy like Burp Suite and noticing unusual headers in responses.

The response reveals a new, previously hidden endpoint: `/hallomessenger`.

Step 5: Gaining Access to the Messenger

Navigate to `/hallomessenger`: You go to `http://localhost:9000/hallomessenger`. This page presents you with a simple form asking for a key. You have a limit of **5 attempts**.

Use the Decoded Key: You enter the key `letmein` discovered in Step 3. Access is granted, and you are redirected to a messaging page.

Step 6: Exploiting XSS to Get the Flag

Discover the Forbidden Endpoint: While exploring the application, you find a reference to or directly attempt to access `http://localhost:9000/hallomessenger/secret`. This endpoint returns a **403 Forbidden** error, indicating it exists but requires special privileges.

Identify the Vulnerability: The messaging functionality on the `/hallomessenger` page is vulnerable to **Cross-Site Scripting (XSS)**. Any message you submit is rendered directly on the page without sanitization.

Craft the XSS Payload: The goal is to make an authenticated request to the forbidden `/secret` endpoint. From the error or further testing, you deduce that accessing `/secret` requires the header `X-Admin: 1`.

You craft a payload that uses JavaScript to fetch the `/secret` page with the required admin header and then display its contents on the page.

Payload:

```
<script>
fetch('/hallomessenger/secret', { headers: {'X-Admin': '1'} })
  .then(response => response.text())
  .then(flagText => {
    // Display the flag on the page
    document.body.insertAdjacentHTML('beforeend', '<pre>FLAG: ' + flagText +
  });
</script>
```

Execute the Attack: You submit this payload into the message box. When the page loads (or refreshes,

depending on the app's logic), the script executes. It makes a request to the secret endpoint with the proper `X-Admin` header, retrieves the flag, and prints it directly onto the messenger page.

Alternative Method (Direct Request): Once you know the required header and endpoint, you can also retrieve the flag directly using `curl` without the XSS:

```
curl -H "X-Admin: 1" http://localhost:9000/hallomessenger/secret
```

This will return the flag directly in your terminal.

Step 7: Capture the Flag

After the XSS payload executes or you run the `curl` command, you receive the flag.

FLAG: HAUNT{********}**